Lernzusammenfassung: Git, Bash, DuckDB, SQL

Session vom 11.11.2026 · Repo DE-learning · Ordner sql-basics

1. Terminal (Bash) und Navigation

`pwd`	Zeigt, in welchem Ordner du gerade bist.
`ls`	Listet die Dateien im aktuellen Ordner.
`cd ordner`	Wechselt in einen Unterordner.
`cd ..`	Eine Ebene nach oben (relativ zum aktuellen Ordner).
`cd ~/Documents/DE-learning`	~ ist der Home-Ordner (/Users/mehmet), nicht "alles davor". Funktioniert von überall aus.
`mkdir name / touch datei`	Ordner bzw. leere Datei anlegen.
`cat > datei << 'EOF' ... EOF`	Schreibt den Text bis EOF in eine Datei. Abkürzung für Tutorials.
`head -5 datei`	Zeigt die ersten 5 Zeilen als Rohtext.
`wc -l datei`	Zählt Zeilen (CSV: Kopfzeile + Datensätze).
`Pfeil hoch`	Holt den letzten Befehl zurück. Enter führt ihn erneut aus.

Wo bin ich? Steht vorn `(base) ...sql-basics mehmet$`, bist du in Bash und tippst Terminal-Befehle. Steht dort `D`, bist du in DuckDB und tippst SQL (Ende mit Semikolon). SQL im Bash-Prompt gibt "command not found".

2. Git und GitHub

`git clone URL`	Holt ein GitHub-Repo als Ordner auf den Rechner. Der Ordnername kommt aus der URL, ohne .git.
`git status`	Zeigt neue und geänderte Dateien. Vor jedem Commit prüfen.
`git add .`	Merkt Dateien für den Commit vor (Warenkorb). Noch nichts gespeichert.
`git commit -m "Text"`	Speichert den Stand als Schnappschuss, nur lokal. Nachricht sagt, was sich geändert hat.
`git push`	Lädt die Commits zu GitHub hoch. Erst danach sichtbar im Browser.
`gh auth login`	Einmaliger Login (Browser-Variante). Ohne Login klappt git push nicht. SSH ist nicht nötig.
`.gitignore`	Dateien, die Git überspringt. Zeile *.duckdb hält die Datenbankdatei aus dem Repo.

Merksatz: add wählt aus, commit speichert lokal, push lädt hoch. Lieber oft in kleinen Schritten committen, mit aussagekräftiger Nachricht (nicht "update"). Git kennt keine leeren Ordner, ein Ordner entsteht mit einer Datei darin.

3. Setup auf dem Mac: Homebrew und VS Code

`brew install git gh duckdb`	Homebrew ist der Paketmanager für macOS. Installiert Kommandozeilen-Tools per Befehl.
`brew install --cask visual-studio-code`	Installiert VS Code direkt in Programme.
`code .`	Öffnet den aktuellen Ordner in VS Code. Braucht einen Link in /usr/local/bin/code.
`ls -l /usr/local/bin/code`	Zeigt, wohin der Link zeigt. Zeigt er ins Leere, liegt die App nicht in Programme.

Was schiefging: VS Code lag in Downloads statt in Programme, deshalb zeigte der Link ins Leere ("code: command not found"). Lösung: alte Apps löschen, frisch per Homebrew installieren. Fehlermeldungen systematisch lesen: Existiert der Link? Existiert das Ziel? Stimmt der PATH?

4. DuckDB

`duckdb lernen.duckdb`	Startet die interaktive Sitzung (Prompt D). Zum Ausprobieren.
`duckdb lernen.duckdb < datei.sql`	Führt eine SQL-Datei aus. < heißt: Datei als Eingabe lesen.
`duckdb -c "SELECT ...;"`	Einzelne Query direkt aus Bash.
`SELECT * FROM 'datei.csv';`	Fragt eine CSV direkt per SQL ab, ohne Datenbank.
`.tables / DESCRIBE tabelle;`	Tabellen bzw. Spalten und Typen anzeigen.
`.exit oder Ctrl + D`	Sitzung beenden, zurück in Bash.
`setup.sql`	Legt Tabellen mit expliziten Typen aus den CSVs an. Quelle der Wahrheit.

lernen.duckdb ist eine lokale Arbeitskopie in einer Datei. CSVs und setup.sql sind die Quelle, die Datei lässt sich jederzeit neu erzeugen (deshalb in .gitignore). **Typen:** DuckDB rät Spaltentypen. Explizite Typen im Setup-Skript sind verlässlicher, z. B. VARCHAR für Status mit führenden Nullen (090 bleibt 090). Vorsicht: CSVs nicht in Excel öffnen, Excel macht aus 090 die Zahl 90.

5. SQL-Konzepte

`SELECT f.vin FROM f_vehicles f`	Alias f ist ein lokaler Name für die Tabelle. SELECT-Liste ändert Spalten, nicht die Zeilenzahl.
`WHERE x = NULL`	Liefert nie Zeilen. Vergleich mit NULL ist UNKNOWN, und WHERE behält nur TRUE.
`WHERE x IS NULL`	Richtiger Test auf fehlende Werte.
`a AND (b OR c IS NULL)`	AND bindet stärker als OR. Klammern halten die Datums-Alternativen zusammen, NULL-Zeilen umgehen dann den Rest nicht.
`CASE WHEN ... THEN ... ELSE ... END`	Gibt den ersten wahren Zweig zurück. Bei NULL ist die Bedingung nicht TRUE, also greift ELSE.
`count(*) vs. count(spalte)`	count(*) zählt alle Zeilen, count(spalte) überspringt NULLs.
`count(DISTINCT order_no_15)`	Zählt eindeutige Werte. Vergleich mit count(*) zeigt Duplikate.
`-- Kommentar`	Alles hinter -- ignoriert SQL. In VS Code: Cmd + /.

Grain (Granularität) sagt, wofür **eine Zeile** steht. f_vehicles: eine Zeile pro Order. te_orders_history: eine Zeile pro Order-Event. Kurzcheck bei jeder Query: Gibt es ein JOIN? Dann können Zeilen mehr werden. Gibt es ein GROUP BY? Dann ist eine Zeile eine Gruppe. Beides nein? Dann bleibt der Grain wie in der Quelle, WHERE entfernt nur Zeilen. Beispiel: Join Snapshot mit Historie ergibt 8 statt 13 Zeilen (Order 1: 3 Events, Order 2: 2, Order 3: 2, Order 4: 1, übrige Orders: 0 bei INNER JOIN).

6. Arbeitsweise eines Data Engineers

`Erkunden`	Interaktiv ausprobieren (hier DuckDB-Sitzung, im Job Athena-Konsole, Snowflake-Worksheet, Databricks-Notebook, DBeaver).
`Festhalten`	Was funktioniert, kommt als .sql-Datei ins Repo: versioniert, nachvollziehbar, wiederholbar. Im Job oft Teil einer Pipeline (dbt-Modell, Job).
`Namen lesen`	datenbank.tabelle in Athena. Präfixe (f_, te_) und Abkürzungen sind Projektkonvention. Beim Kollegen erfragen und als Glossar in die README.
`Session-Ende`	Alles speichern, .exit, git status prüfen, add / commit / push, Stand und nächsten Schritt in die README schreiben.

Nächste Schritte: Sektion 04 (CASE) mit Order 12 (order_type NULL) und dem Test auf ivs_order_status, Sektion 03 (CTE-Kette) von Hand in uebung03.sql, Präfixe f_ und te_ beim Kollegen erfragen. Hinweis: sequence(...) aus der Athena-Übung heißt in DuckDB generate_series(...).